

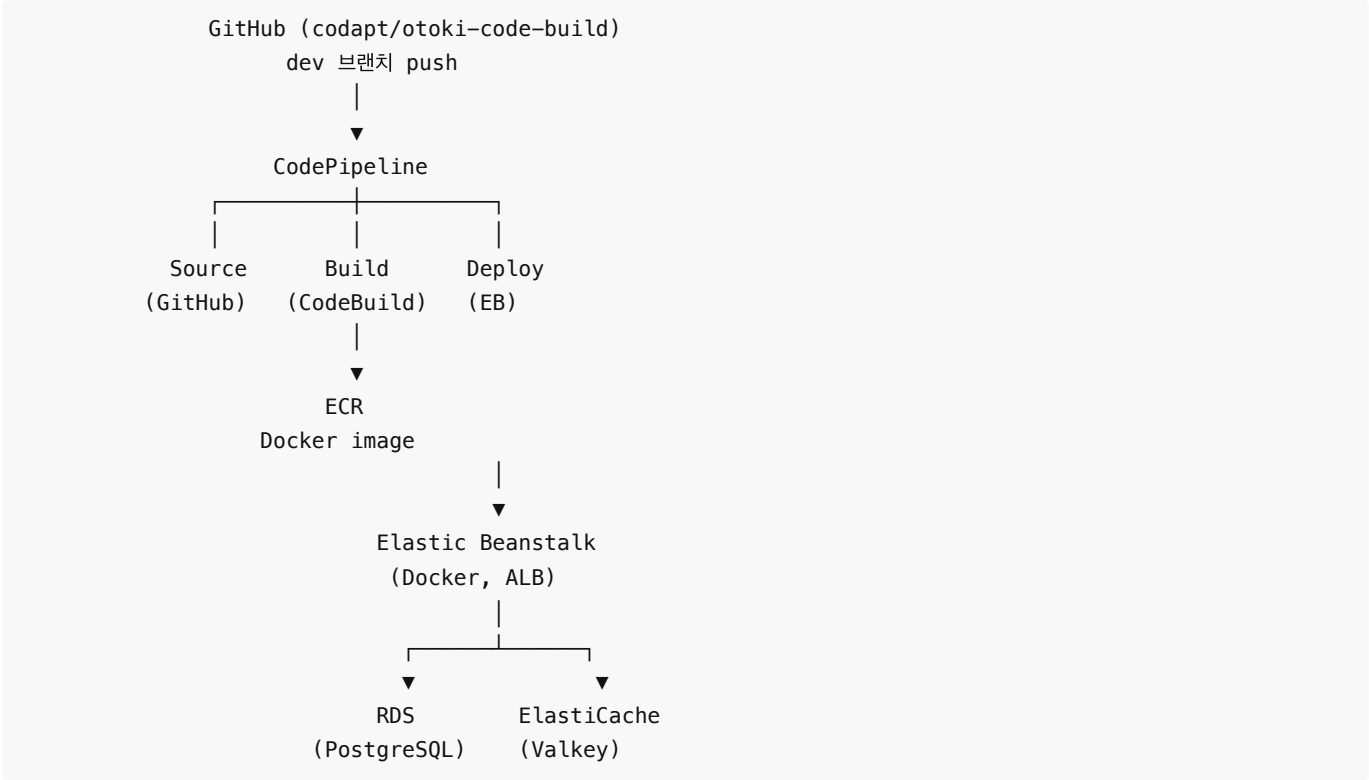
otoki-code-build AWS 인프라 가이드 — Backend

0. 전체 아키텍처

프로젝트 개요

항목	내용
저장소	GitHub codapt/otoki-code-build (모노레포)
Backend	Kotlin + Spring Boot 4.0.5 (Java 24)
리전	ap-northeast-2 (서울)
Stage	dev, prod

아키텍처 다이어그램



		(Valkey 8.2)	
--	--	--------------	--

항목별 작업자 배정

섹션	항목	작업자	비고
2	VPC & 네트워킹	인프라	모든 리소스의 기반
3	IAM (역할 생성)	인프라	EB Service/EC2 Role, CodeBuild Role
4	RDS (PostgreSQL)	인프라	VPC, Security Group 필요
5	ElastiCache (Valkey)	인프라	VPC, Security Group 필요
6	ACM 인증서	인프라	HTTPS용 SSL 인증서
7.1	GitHub Connection	앱	Organization Owner/Admin 권한 필요
7.2	CodePipeline Role	인프라	
8	ECR	앱	
9	Elastic Beanstalk	앱	인프라 작업자로부터 VPC, IAM, RDS/Redis endpoint 전달받아 사용
10	CI/CD (Backend)	앱	CodePipeline + CodeBuild
11	DNS 레코드	인프라	EB URL을 Route 53에 등록
12	CloudWatch	앱	모든 리소스 생성 후
13	DB 접속 (SSM)	앱	EB 인스턴스 경유 포트 포워딩
14	Dev vs Prod 요약	공통	참고용

앱 작업자 IAM 권한

AWS 관리형 정책 (6개):

정책	용도
AdministratorAccess-AWSElasticBeanstalk	EB 전체 관리 (섹션 9)
AmazonEC2ContainerRegistryFullAccess	ECR 리포지토리 관리 (섹션 8)
AWSCodeBuildAdminAccess	CodeBuild 프로젝트 관리 (섹션 10)
AWSCodePipeline_FullAccess	CodePipeline + GitHub Connection 관리 (섹션 7.1, 10)
AmazonVPCReadOnlyAccess	EB 생성 시 VPC/서브넷 조회 (섹션 9)
CloudWatchFullAccess	CloudWatch 알람 + 로그 관리 (섹션 12)

인라인 정책 (1개, 이름: `dev-otk-pwrs-app-developer-policy`):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
```

```

    "Sid": "SNSTopic",
    "Effect": "Allow",
    "Action": [
        "sns:CreateTopic",
        "sns:Subscribe",
        "sns:SetTopicAttributes",
        "sns:TagResource"
    ],
    "Resource": "arn:aws:sns:ap-northeast-2:*:*-otk-pwrs-*"
},
{
    "Sid": "S3PipelineArtifacts",
    "Effect": "Allow",
    "Action": "s3:*",
    "Resource": [
        "arn:aws:s3:::*-otk-pwrs-pipeline-artifacts",
        "arn:aws:s3:::*-otk-pwrs-pipeline-artifacts/*"
    ]
},
{
    "Sid": "SSMSessionStart",
    "Effect": "Allow",
    "Action": [
        "ssm:StartSession",
        "ssm:TerminateSession",
        "ssm:ResumeSession"
    ],
    "Resource": [
        "arn:aws:ec2:ap-northeast-2:*:instance/*",
        "arn:aws:ssm:*:*:document/AWS-StartPortForwardingSessionToRemoteHost"
    ],
    "Condition": {
        "StringEquals": {
            "aws:ResourceTag/Project": "otk-pwrs"
        }
    }
},
{
    "Sid": "SSMDescribe",
    "Effect": "Allow",
    "Action": [
        "ssm:DescribeSessions",
        "ssm:DescribeInstanceInformation"
    ],
    "Resource": "*"
},
{
    "Sid": "EC2Describe",
    "Effect": "Allow",
    "Action": "ec2:DescribeInstances",
    "Resource": "*"
},
{
    "Sid": "CodeConnections",
    "Effect": "Allow",
    "Action": "codeconnections:*",

```

```

    "Resource": "*"
  },
  {
    "Sid": "PassRole",
    "Effect": "Allow",
    "Action": "iam:PassRole",
    "Resource": [
      "arn:aws:iam::983241034734:role/*-otk-pwrs-*"
    ]
  }
]
}

```

Route 53 DNS 레코드 등록(섹션 11)은 앱 작업자가 EB URL을 전달하면 인프라 작업자가 처리한다.

셋업 순서

인프라 작업자와 앱 작업자가 분리되어 있다. 의존성에 따라 아래 순서대로 진행한다.

[인프라 작업자]

1. VPC & 네트워킹 ← 모든 리소스의 기반
2. IAM (역할 생성) ← EB Service/EC2 Role, CodeBuild Role
3. RDS (PostgreSQL) ← VPC, Security Group 필요
4. ElastiCache (Valkey) ← VPC, Security Group 필요
5. ACM 인증서 ← HTTPS용 SSL 인증서

[앱 작업자 → 인프라 작업자]

6. GitHub Connection ← 앱 작업자가 생성
7. CodePipeline Role ← 인프라 작업자가 인라인 정책 생성

[앱 작업자]

8. ECR
9. Elastic Beanstalk ← VPC, IAM, RDS/Redis endpoint, ECR 이미지 필요
10. CI/CD (Backend) ← CodePipeline + CodeBuild (EB 앱/환경 이름 필요)

[앱 작업자 → 인프라 작업자]

11. DNS 레코드 ← 앱 작업자가 EB URL을 인프라 작업자에게 전달, 인프라 작업자가 Route 53에 등록

[앱 작업자]

12. CloudWatch ← 모든 리소스 생성 후
13. DB 접속 (SSM) ← EB 인스턴스 경유 포트 포워딩

1. 네이밍 컨벤션 & 태깅

패턴

```
{stage}-otk-pwrs-{service}
```

예: dev-otk-pwrs-backend-eb , prod-otk-pwrs-rds

리소스 이름 전체 목록

리소스	Dev	Prod
VPC	dev-otk-pwrs-vpc	prod-otk-pwrs-vpc

Public Subnet AZ-a	dev-otk-pwrs-public-a	prod-otk-pwrs-public-a
Public Subnet AZ-c	dev-otk-pwrs-public-c	prod-otk-pwrs-public-c
App Subnet AZ-a	dev-otk-pwrs-app-a	prod-otk-pwrs-app-a
App Subnet AZ-c	dev-otk-pwrs-app-c	prod-otk-pwrs-app-c
Data Subnet AZ-a	dev-otk-pwrs-data-a	prod-otk-pwrs-data-a
Data Subnet AZ-c	dev-otk-pwrs-data-c	prod-otk-pwrs-data-c
Internet Gateway	dev-otk-pwrs-igw	prod-otk-pwrs-igw
NAT Gateway	dev-otk-pwrs-nat	prod-otk-pwrs-nat
Route Table (Public)	dev-otk-pwrs-public-rt	prod-otk-pwrs-public-rt
Route Table (App)	dev-otk-pwrs-app-rt	prod-otk-pwrs-app-rt
Route Table (Data)	dev-otk-pwrs-data-rt	prod-otk-pwrs-data-rt
NACL (App)	dev-otk-pwrs-app-nacl	prod-otk-pwrs-app-nacl
NACL (Data)	dev-otk-pwrs-data-nacl	prod-otk-pwrs-data-nacl
SG - ALB	dev-otk-pwrs-alb-sg	prod-otk-pwrs-alb-sg
SG - EB 인스턴스	dev-otk-pwrs-eb-sg	prod-otk-pwrs-eb-sg
SG - RDS	dev-otk-pwrs-rds-sg	prod-otk-pwrs-rds-sg
SG - ElastiCache	dev-otk-pwrs-redis-sg	prod-otk-pwrs-redis-sg
RDS 인스턴스	dev-otk-pwrs-db	prod-otk-pwrs-db
RDS 서브넷 그룹	dev-otk-pwrs-db-subnet-group	prod-otk-pwrs-db-subnet-group
ElastiCache 클러스터	dev-otk-pwrs-redis	prod-otk-pwrs-redis
ElastiCache 서브넷 그룹	dev-otk-pwrs-redis-subnet-group	prod-otk-pwrs-redis-subnet-group
ECR	dev-otk-pwrs	prod-otk-pwrs
EB Application	dev-otk-pwrs-backend	prod-otk-pwrs-backend
EB Environment	dev-otk-pwrs-backend-env	prod-otk-pwrs-backend-env
CodeBuild (Backend)	dev-otk-pwrs-backend-build	prod-otk-pwrs-backend-build
CodePipeline (Backend)	dev-otk-pwrs-backend-pipeline	prod-otk-pwrs-backend-pipeline
GitHub 연결 (Connection)	otk-pwrs-github	(동일, 공유)
S3 (Pipeline Artifacts)	dev-otk-pwrs-pipeline-artifacts	prod-otk-pwrs-pipeline-artifacts
S3 (Storage)	dev-otk-pwrs-storage	prod-otk-pwrs-storage
IAM - EB Service Role	aws-elasticbeanstalk-service-role	(동일, AWS managed)
IAM - EB Instance Profile	dev-otk-pwrs-eb-ec2-role	prod-otk-pwrs-eb-ec2-role
IAM - CodeBuild Role (Backend)	dev-otk-pwrs-backend-build-role	prod-otk-pwrs-backend-build-role

IAM - CodePipeline Role	dev-otk-pwrs-pipeline-role	prod-otk-pwrs-pipeline-role
SNS 알림 토픽	(미설정)	prod-otk-pwrs-alerts

태그 정책

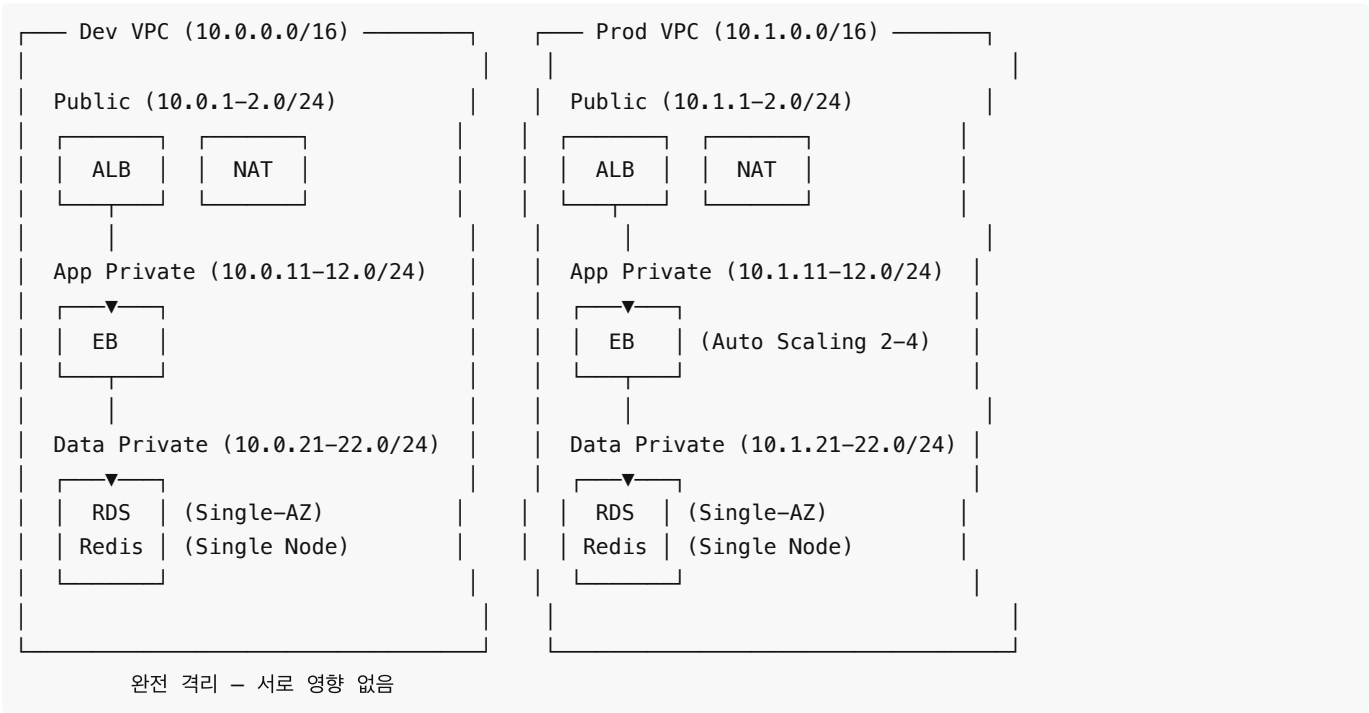
모든 리소스에 아래 태그를 부여한다:

태그 키	값
Stage	dev 또는 prod
Project	otk-pwrs
ManagedBy	terraform

인프라 작업자 영역

2. VPC & 네트워킹

dev/prod 각각 독립된 VPC를 사용한다 (3-Tier 서버넷, VPC 분리).



2.1 VPC 생성

콘솔: VPC > Your VPCs > Create VPC

설정	Dev	Prod
Name	dev-otk-pwrs-vpc	prod-otk-pwrs-vpc
IPv4 CIDR	10.0.0.0/16	10.1.0.0/16
IPv6	없음	없음
Tenancy	Default	Default

CIDR을 분리하면 향후 VPC Peering 시 충돌이 없다.

2.2 서브넷 생성 (3-Tier)

콘솔: VPC > Subnets > Create subnet

Dev 서브넷 (`dev-otk-pwrs-vpc` 내):

Name	AZ	CIDR	계층	용도
dev-otk-pwrs-public-a	ap-northeast-2a	10.0.1.0/24	Public	ALB, NAT GW
dev-otk-pwrs-public-c	ap-northeast-2c	10.0.2.0/24	Public	ALB
dev-otk-pwrs-app-a	ap-northeast-2a	10.0.11.0/24	App Private	EB 인스턴스
dev-otk-pwrs-app-c	ap-northeast-2c	10.0.12.0/24	App Private	EB 인스턴스
dev-otk-pwrs-data-a	ap-northeast-2a	10.0.21.0/24	Data Private	RDS
dev-otk-pwrs-data-c	ap-northeast-2c	10.0.22.0/24	Data Private	RDS

Prod 서브넷 (`prod-otk-pwrs-vpc` 내):

Name	AZ	CIDR	계층	용도
prod-otk-pwrs-public-a	ap-northeast-2a	10.1.1.0/24	Public	ALB, NAT GW
prod-otk-pwrs-public-c	ap-northeast-2c	10.1.2.0/24	Public	ALB, NAT GW
prod-otk-pwrs-app-a	ap-northeast-2a	10.1.11.0/24	App Private	EB 인스턴스
prod-otk-pwrs-app-c	ap-northeast-2c	10.1.12.0/24	App Private	EB 인스턴스
prod-otk-pwrs-data-a	ap-northeast-2a	10.1.21.0/24	Data Private	RDS
prod-otk-pwrs-data-c	ap-northeast-2c	10.1.22.0/24	Data Private	RDS

CIDR 배치 규칙:

Dev (10.0.0.0/16)	Prod (10.1.0.0/16)
├─ Public: 10.0.1-2.0/24	├─ Public: 10.1.1-2.0/24
├─ App: 10.0.11-12.0/24	├─ App: 10.1.11-12.0/24
└─ Data: 10.0.21-22.0/24	└─ Data: 10.1.21-22.0/24

두 VPC의 서브넷 번호 체계가 동일하여 직관적. 두 번째 옥텟(0 vs 1)으로 stage 구분.

2.3 Internet Gateway

콘솔: VPC > Internet Gateways > Create internet gateway

설정	Dev	Prod
Name	dev-otk-pwrs-igw	prod-otk-pwrs-igw
Attach to VPC	dev-otk-pwrs-vpc	prod-otk-pwrs-vpc

VPC당 IGW 1개 필요. IGW 자체는 무료.

2.4 NAT Gateway

콘솔: VPC > NAT Gateways > Create NAT gateway

Name	Subnet	용도
dev-otk-pwrs-nat	dev-otk-pwrs-public-a	Dev App 서브넷용
prod-otk-pwrs-nat	prod-otk-pwrs-public-a	Prod App 서브넷용

각 NAT Gateway에 **Elastic IP**를 새로 할당한다.

Dev/Prod 모두 NAT 1개로 운영한다 (~\$32/월). AZ 단독 장애 시에는 아래 장애 대응 절차로 대응한다.

외부 서비스 연동 (SAP 등) 시 IP 화이트리스트:

App Private 서브넷의 EB 인스턴스가 외부 서비스로 요청할 때, NAT Gateway의 EIP가 소스 IP로 사용된다. Auto Scaling으로 인스턴스가 증감하거나 교체되어도 NAT Gateway EIP는 변동 없으므로, 외부 서비스에 NAT Gateway의 EIP를 등록하면 된다.

NAT Gateway를 삭제/재생성하면 EIP가 변경될 수 있으므로, 변경 시 외부 서비스 화이트리스트도 업데이트해야 한다.

AZ 장애 시 NAT Gateway 대응 절차:

NAT Gateway가 위치한 AZ에 장애가 발생하면 아웃바운드 트래픽(SAP 연동, ECR pull 등)이 중단된다. 아래 절차로 복구한다 (약 10~15분 소요).

1. 정상 AZ의 Public 서브넷에 NAT Gateway 신규 생성 + EIP 할당
2. App 라우팅 테이블의 **0.0.0.0/0** 타겟을 새 NAT Gateway로 변경
3. 아웃바운드 복구 확인
4. 외부 서비스(SAP 등) 화이트리스트에 새 EIP 추가 등록

2.5 라우팅 테이블

콘솔: VPC > Route Tables > Create route table

dev와 prod 각각 동일한 구조로 생성. VPC가 다르므로 이름만 stage 구분.

Dev 라우팅 테이블

dev-otk-pwrs-public-rt :

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	dev-otk-pwrs-igw

→ dev-otk-pwrs-public-a , dev-otk-pwrs-public-c 연결

dev-otk-pwrs-app-rt :

Destination	Target
10.0.0.0/16	local
0.0.0.0/0	dev-otk-pwrs-nat

→ dev-otk-pwrs-app-a , dev-otk-pwrs-app-c 연결

dev-otk-pwrs-data-rt :

Destination	Target
10.0.0.0/16	local

→ dev-otk-pwrs-data-a , dev-otk-pwrs-data-c 연결

Data 계층은 인터넷 아웃바운드 없음. VPC 내부 통신만 허용.

Prod 라우팅 테이블

prod-otk-pwrs-public-rt :

Destination	Target
10.1.0.0/16	local
0.0.0.0/0	prod-otk-pwrs-igw

→ prod-otk-pwrs-public-a , prod-otk-pwrs-public-c 연결

prod-otk-pwrs-app-rt :

Destination	Target
10.1.0.0/16	local
0.0.0.0/0	prod-otk-pwrs-nat

→ prod-otk-pwrs-app-a , prod-otk-pwrs-app-c 연결

prod-otk-pwrs-data-rt :

Destination	Target
10.1.0.0/16	local

→ prod-otk-pwrs-data-a , prod-otk-pwrs-data-c 연결

2.6 NACL (Network ACL)

VPC가 분리되어 있으므로 cross-stage 접근이 원천적으로 불가능하다. NACL은 계층 간 방어에만 집중하면 된다.

콘솔: VPC > Network ACLs > Create network ACL

dev/prod 동일 구조. CIDR만 해당 VPC에 맞게 설정.

App NACL ({stage}-otk-pwrs-app-nacl)

→ App 서브넷 2개에 연결

Dev 기준 (dev-otk-pwrs-app-nacl):

방향	Rule#	프로토콜	포트	소스/대상	허용
Inbound	100	TCP	80	10.0.1.0/24 (public-a)	Allow
Inbound	110	TCP	80	10.0.2.0/24 (public-c)	Allow
Inbound	200	TCP	1024-65535	0.0.0.0/0	Allow
Inbound	*	All	All	0.0.0.0/0	Deny
Outbound	100	TCP	5432	10.0.21.0/24 (data-a)	Allow
Outbound	110	TCP	5432	10.0.22.0/24 (data-c)	Allow

Outbound	120	TCP	6379	10.0.21.0/24 (data-a)	Allow
Outbound	130	TCP	6379	10.0.22.0/24 (data-c)	Allow
Outbound	200	TCP	443	0.0.0.0/0	Allow
Outbound	210	TCP	1024-65535	0.0.0.0/0	Allow
Outbound	*	All	All	0.0.0.0/0	Deny

Inbound 200: NACL은 stateless이므로 NAT Gateway 응답(ephemeral ports)을 명시 허용. Outbound 200: ECR pull, 외부 API 호출용.

Prod: 동일 구조, CIDR을 10.1.x.0/24 로 변경.

Data NACL ({stage}-otk-pwrs-data-nacl)

→ Data 서브넷 2개에 연결

Dev 기준 (dev-otk-pwrs-data-nacl):

방향	Rule#	프로토콜	포트	소스/대상	허용
Inbound	100	TCP	5432	10.0.11.0/24 (app-a)	Allow
Inbound	110	TCP	5432	10.0.12.0/24 (app-c)	Allow
Inbound	120	TCP	6379	10.0.11.0/24 (app-a)	Allow
Inbound	130	TCP	6379	10.0.12.0/24 (app-c)	Allow
Inbound	*	All	All	0.0.0.0/0	Deny
Outbound	100	TCP	1024-65535	10.0.11.0/24 (app-a)	Allow
Outbound	110	TCP	1024-65535	10.0.12.0/24 (app-c)	Allow
Outbound	*	All	All	0.0.0.0/0	Deny

App 서브넷에서 오는 5432(RDS), 6379(Redis) 포트만 허용. 인터넷 아웃바운드 완전 차단.

Prod: 동일 구조, CIDR을 10.1.x.0/24 로 변경.

2.7 Security Groups

콘솔: VPC > Security Groups > Create security group

각 VPC에 동일한 구조로 SG를 생성한다. VPC가 다르므로 SG는 자동으로 격리된다.

ALB SG ({stage}-otk-pwrs-alb-sg)

방향	프로토콜	포트	소스
Inbound	TCP	80	0.0.0.0/0
Inbound	TCP	443	0.0.0.0/0
Outbound	All	All	0.0.0.0/0

EB SG ({stage}-otk-pwrs-eb-sg)

방향	프로토콜	포트	소스
Inbound	TCP	80	{stage}-otk-pwrs-alb-sg

Outbound	All	All	0.0.0.0/0
----------	-----	-----	-----------

RDS SG ({stage}-otk-pwrs-rds-sg)

방향	프로토콜	포트	소스
Inbound	TCP	5432	{stage}-otk-pwrs-eb-sg
Outbound	All	All	0.0.0.0/0

ElastiCache SG ({stage}-otk-pwrs-redis-sg)

방향	프로토콜	포트	소스
Inbound	TCP	6379	{stage}-otk-pwrs-eb-sg
Outbound	All	All	0.0.0.0/0

Security Group 체인: ALB → EB → RDS / ElastiCache. VPC 분리이므로 cross-stage 참조가 물리적으로 불가능.

3. IAM 역할 & 정책 (Backend)

3.1 EB Service Role (aws-elasticbeanstalk-service-role)

EB 환경 생성(9.2) 시 서비스 역할을 선택해야 한다. **사전에 생성해 둔다.**

콘솔: IAM > Roles > Create role

1. Trusted entity: **AWS service** → **Elastic Beanstalk**
2. Use case: **Elastic Beanstalk**
3. 자동 연결되는 정책:
 - AWSElasticBeanstalkManagedUpdatesCustomerRolePolicy
 - AWSElasticBeanstalkEnhancedHealth
4. Role name: **aws-elasticbeanstalk-service-role**
5. 인라인 정책 추가 (이름: **eb-service-pipeline-artifacts-policy**):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PipelineArtifacts",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:GetObjectAcl",
        "s3:GetBucketLocation",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts",
        "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts/*",
        "arn:aws:s3:::prod-otk-pwrs-pipeline-artifacts",
        "arn:aws:s3:::prod-otk-pwrs-pipeline-artifacts/*"
      ]
    }
  ]
}
```

```
]
}
```

CodePipeline Deploy 스테이지에서 EB 서비스 역할이 Pipeline Artifacts 버킷의 아티팩트를 읽어야 한다. 읽기 전용이므로 Put 계열 권한은 불필요.

3.2 EB EC2 Instance Profile (dev-otk-pwrs-eb-ec2-role)

콘솔: IAM > Roles > Create role

1. Trusted entity: **AWS service** → **EC2**
2. Role name: dev-otk-pwrs-eb-ec2-role
3. 연결할 정책:
 - AWSElasticBeanstalkWebTier
 - AmazonEC2ContainerRegistryReadOnly (ECR에서 이미지 pull)
 - CloudWatchAgentServerPolicy
 - AmazonSSMManagedInstanceCore (Session Manager를 통한 인스턴스 접속)
4. 인라인 정책 추가 (이름: dev-otk-pwrs-eb-ec2-storage-policy):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "S3Storage",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::dev-otk-pwrs-storage",
        "arn:aws:s3:::dev-otk-pwrs-storage/*"
      ]
    }
  ]
}
```

AWSElasticBeanstalkWebTier 는 elasticbeanstalk-* 버킷만 포함하므로, 앱 Storage 버킷은 별도 인라인 정책으로 허용한다. Prod 역할에서는 Resource를 prod-otk-pwrs-storage 로 변경한다.

5. 인라인 정책 추가 (이름: dev-otk-pwrs-eb-ec2-app-config-policy):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ParameterStoreRead",
      "Effect": "Allow",
      "Action": [
        "ssm:GetParameter",
        "ssm:GetParameters",
        "ssm:GetParametersByPath"
      ],
    }
  ]
}
```

```

    "Resource": "arn:aws:ssm:ap-northeast-2:<ACCOUNT_ID>:parameter/otk-pwrs/dev/*"
  },
  {
    "Sid": "RDSSecretRead",
    "Effect": "Allow",
    "Action": [
      "secretsmanager:GetSecretValue",
      "secretsmanager:DescribeSecret"
    ],
    "Resource": "<RDS_MASTER_USER_SECRET_ARN>"
  },
  {
    "Sid": "KmsDecryptRDSSecret",
    "Effect": "Allow",
    "Action": "kms:Decrypt",
    "Resource": "<RDS_MASTER_USER_SECRET_KMS_KEY_ARN>"
  }
]
}

```

런타임에 앱이 RDS master user 비밀번호를 Secrets Manager 에서 직접 조회할 때 사용한다. 비-비밀 값(RDS/Redis endpoint 등)은 Parameter Store 경로에서 읽는다. 경로·ARN·KMS 키는 섹션 "앱 구성 전달 (SSM Parameter Store)"에 publish 된 값을 그대로 사용한다. Prod 역할에서는 Resource 경로를 `parameter/otk-pwrs/prod/*` , secret ARN을 prod RDS 인스턴스의 것으로 변경한다.

3.3 Backend CodeBuild Role (dev-otk-pwrs-backend-build-role)

콘솔: IAM > Roles > Create role

1. Trusted entity: **AWS service** → **CodeBuild**
2. Role name: `dev-otk-pwrs-backend-build-role`
3. 권한 정책 선택: 건너뛰기 (아무 정책도 선택하지 않음)
4. 역할 생성 후 > 권한 탭 > 권한 추가 > 인라인 정책 생성 > JSON
5. 인라인 정책 이름: `dev-otk-pwrs-backend-build-policy`

CodePipeline을 사용하므로 CodeBuild는 Docker 빌드 + ECR Push만 담당한다. EB 배포 권한은 불필요.

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "ECRAuth",
      "Effect": "Allow",
      "Action": "ecr:GetAuthorizationToken",
      "Resource": "*"
    },
    {
      "Sid": "ECRPush",
      "Effect": "Allow",
      "Action": [
        "ecr:BatchCheckLayerAvailability",
        "ecr:GetDownloadUrlForLayer",
        "ecr:BatchGetImage",
        "ecr:PutImage",
        "ecr:InitiateLayerUpload",
        "ecr:UploadLayerPart",
        "ecr:CompleteLayerUpload"
      ],
    },
  ],
}

```

```

    "Resource": "arn:aws:ecr:ap-northeast-2:<ACCOUNT_ID>:repository/dev-otk-pwrs"
  },
  {
    "Sid": "PipelineArtifacts",
    "Effect": "Allow",
    "Action": [
      "s3:GetObject",
      "s3:PutObject",
      "s3:GetBucketAcl",
      "s3:GetBucketLocation"
    ],
    "Resource": [
      "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts",
      "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts/*"
    ]
  },
  {
    "Sid": "EBBucket",
    "Effect": "Allow",
    "Action": [
      "s3:PutObject",
      "s3:GetObject",
      "s3:GetBucketLocation"
    ],
    "Resource": [
      "arn:aws:s3:::elasticbeanstalk-ap-northeast-2-<ACCOUNT_ID>",
      "arn:aws:s3:::elasticbeanstalk-ap-northeast-2-<ACCOUNT_ID>/*"
    ]
  },
  {
    "Sid": "CloudWatchLogs",
    "Effect": "Allow",
    "Action": [
      "logs:CreateLogGroup",
      "logs:CreateLogStream",
      "logs:PutLogEvents"
    ],
    "Resource": "arn:aws:logs:ap-northeast-2:<ACCOUNT_ID>:log-group:/aws/codebuild/*"
  },
  {
    "Sid": "CodeConnections",
    "Effect": "Allow",
    "Action": [
      "codeconnections:UseConnection",
      "codeconnections:GetConnection",
      "codeconnections:GetConnectionToken",
      "codestar-connections:UseConnection",
      "codestar-connections:GetConnection",
      "codestar-connections:GetConnectionToken"
    ],
    "Resource": "arn:aws:codeconnections:ap-northeast-2:<ACCOUNT_ID>:connection/*"
  },
  {
    "Sid": "AppConfigRead",
    "Effect": "Allow",
    "Action": [

```

```

    "ssm:GetParameter",
    "ssm:GetParameters",
    "ssm:GetParametersByPath"
  ],
  "Resource": "arn:aws:ssm:ap-northeast-2:<ACCOUNT_ID>:parameter/otk-pwrs/dev/*"
}
]
}

```

`AppConfigRead` 는 `buildspec` 이 `.ebextensions/env.config` 를 생성할 때 SSM Parameter Store 에서 RDS/Redis endpoint 등을 읽기 위해 필요하다. 상세는 [섹션 "앱 구성 전달 \(SSM Parameter Store\)"](#) 참고.

4. RDS (PostgreSQL 17.6)

4.1 DB 서브넷 그룹 생성

콘솔: RDS > Subnet groups > Create DB subnet group

설정	값
Name	dev-otk-pwrs-db-subnet-group
VPC	dev-otk-pwrs-vpc
Subnets	dev-otk-pwrs-data-a, dev-otk-pwrs-data-c (Data Private 서브넷)

4.2 RDS 인스턴스 생성

콘솔: RDS > Databases > Create database

설정	Dev	Prod
생성 방식	Standard create	Standard create
Engine	PostgreSQL 17.6	PostgreSQL 17.6
Template	Free tier	Production
DB instance ID	dev-otk-pwrs-db	prod-otk-pwrs-db
Master username	otkadmin	otkadmin
Master password	AWS Secrets Manager 자동 관리 (Manage master credentials in AWS Secrets Manager 체크)	동일
Instance class	db.t4g.micro	db.t4g.small
Storage	20 GB gp3	50 GB gp3 + autoscaling
Multi-AZ	No	No (필요 시 콘솔에서 전환 가능)
VPC	dev-otk-pwrs-vpc	prod-otk-pwrs-vpc
Subnet group	dev-otk-pwrs-db-subnet-group	prod-otk-pwrs-db-subnet-group
Public access	No	No
Security group	dev-otk-pwrs-rds-sg	prod-otk-pwrs-rds-sg

Initial database name	otoki	otoki
Backup retention	3일	14일
Deletion protection	Off	On
Performance Insights	On (Free tier)	On

4.3 RDS 엔드포인트 & 비밀번호 전달

생성 완료 후 **Connectivity & security** 탭에서 Endpoint와 **Manage master credentials in AWS Secrets Manager** 블록의 Secret ARN을 메모한다:

```
Endpoint:    dev-otk-pwrs-db.xxxxxxxxxx.ap-northeast-2.rds.amazonaws.com
Secret ARN:  arn:aws:secretsmanager:ap-northeast-2:<ACCOUNT_ID>:secret:rds!db-xxxxxxx-xxxx-xxxx-xxxx-
xxxxxxxxxxx-xxxxxx
```

→ 이 값들은 섹션 "앱 구성 전달 (SSM Parameter Store)" 에 따라 Parameter Store 에 publish 되고, EB 배포 시 `.ebextensions/env.config` 를 통해 `RDS_HOST` , `RDS_SECRET_ARN` 등 env var 로 자동 주입된다. 수동 전달은 필요 없다.

4.4 Pipeline Artifact S3 버킷 생성

콘솔: S3 > Create bucket

설정	Dev	Prod
Bucket name	dev-otk-pwrs-pipeline-artifacts	prod-otk-pwrs-pipeline-artifacts
Region	ap-northeast-2	ap-northeast-2
Object Ownership	ACLs disabled (BucketOwnerEnforced)	ACLs disabled (BucketOwnerEnforced)
Versioning	Enabled	Enabled
Encryption	SSE-S3	SSE-S3

파이프라인 스테이지 간 아티팩트(소스 코드, 빌드 결과물)를 전달하는 버킷이다. **Object Ownership**: AWS 권장 기본값 (`BucketOwnerEnforced`)을 사용한다. CodePipeline → EB 배포는 IAM 역할로 접근하므로 ACL이 필요하지 않다.

4.5 Storage S3 버킷 생성

콘솔: S3 > Create bucket

설정	Dev	Prod
Bucket name	dev-otk-pwrs-storage	prod-otk-pwrs-storage
Region	ap-northeast-2	ap-northeast-2
Object Ownership	ACLs disabled	ACLs disabled
Block Public Access	Block all	Block all
Versioning	Disabled	Enabled
Encryption	SSE-S3	SSE-S3

애플리케이션에서 사용하는 이미지, 문서 등 파일 저장용 버킷이다. EB EC2 역할(3.2)에 이 버킷에 대한 읽기/쓰기 권한이 포함되어 있다.

5. ElastiCache (Valkey)

5.1 ElastiCache 서브넷 그룹 생성

콘솔: ElastiCache > Subnet groups > Create subnet group

설정	값
Name	dev-otk-pwrs-redis-subnet-group
VPC	dev-otk-pwrs-vpc
Subnets	dev-otk-pwrs-data-a, dev-otk-pwrs-data-c (Data Private 서브넷)

Prod: `prod-otk-pwrs-redis-subnet-group` , VPC와 서브넷을 `prod`로 변경.

5.2 Valkey 캐시 생성

콘솔: ElastiCache > Valkey caches > Create Valkey cache

구성

설정	Dev	Prod
엔진	Valkey	Valkey
배포 옵션	노드 기반 캐시	노드 기반 캐시
생성 방법	간편한 생성	간편한 생성
구성 프리셋	데모	데모

간편한 생성 시 `Cluster mode`는 자동으로 `Disabled`된다 (별도 설정 불필요).

구성 프리셋 참고:

프리셋	노드 타입	메모리	네트워크
프로덕션	<code>cache.r7g.xlarge</code>	26.32 GiB	Up to 12.5 Gigabit
개발 및 테스트	<code>cache.r7g.large</code>	13.07 GiB	Up to 12.5 Gigabit
데모	<code>cache.t4g.micro</code>	0.5 GiB	Up to 5 Gigabit

Dev/Prod 모두 `데모` (`cache.t4g.micro` , 0.5 GiB, 월 ~\$10)로 시작한다. 세션 + 캐싱 용도로 초기에는 충분하며, 트래픽 증가 시 콘솔에서 노드 타입을 변경할 수 있다.

클러스터 정보

설정	Dev	Prod
이름	dev-otk-pwrs-redis	prod-otk-pwrs-redis

연결

설정	Dev	Prod
네트워크 유형	IPv4	IPv4
서브넷 그룹	dev-otk-pwrs-redis-subnet-group	prod-otk-pwrs-redis-subnet-group

기존 서브넷 그룹 선택 → 5.1에서 생성한 서브넷 그룹을 선택한다. 연결된 서브넷이 Data Private 서브넷 2개 (`data-a` : 10.0.21.0/24, `data-c` : 10.0.22.0/24) 인지 확인한다.

나머지 설정 (간편한 생성 기본값 확인)

설정	Dev	Prod
Security group	dev-otk-pwrs-redis-sg	prod-otk-pwrs-redis-sg
Encryption in-transit	No	No
Encryption at-rest	Yes	Yes

TLS (in-transit) 미사용 이유: VPC 내부(EB ↔ ElastiCache) 통신만 허용되며 Security Group 으로 접근이 제한되므로 암호화 없이도 안전하다. Spring Data Redis 클라이언트(Lettuce) 의 TLS 구성 복잡도를 피하고 표준 평문 프로토콜을 사용한다.

Valkey 엔진 선택 이유:

- Redis OSS v7.0과 완벽 호환 (Linux Foundation 오픈소스 포크).
- 노드 기반 캐시에서 Redis OSS 대비 최대 20% 비용 절감.
- Spring Data Redis가 그대로 동작한다 (프로토콜 호환).

노드 기반 캐시 선택 이유:

- 서버리스는 최소 베이스라인 비용이 ~\$90/월이므로 초기 단계에서 비용 부담.
- 노드 기반 데모 프리셋: 월 ~\$10로 시작 가능.
- 트래픽이 안정적이고 예측 가능한 경우 노드 기반이 비용 효율적.

단일 노드 유의사항:

- 노드 장애 시 데이터가 유실된다. 세션 스토어 특성상 재로그인으로 복구 가능하며, 캐시는 애플리케이션이 자동으로 재구축한다.
- Prod 트래픽 증가 시 `cache.r7g.large` (13.07 GiB)로 업그레이드 가능.

5.3 Redis 엔드포인트 전달

생성 완료 후 **Configuration** 탭에서 **Primary endpoint** (cluster mode disabled 이므로) 와 포트를 메모한다:

```
dev-otk-pwrs-redis.xxxxxx.apn2.cache.amazonaws.com:6379
```

→ 이 값은 섹션 "앱 구성 전달 (SSM Parameter Store)" 에 따라 Parameter Store 에 publish 되고, EB 배포 시 `.ebextensions/env.config` 를 통해 `REDIS_ENDPOINT` , `REDIS_PORT` env var 로 자동 주입된다.

앱 구성 전달 (SSM Parameter Store)

Terraform 이 생성하는 동적 값(RDS/Redis endpoint 등) 과 stage 에 따라 달라지는 값을 Spring Boot 컨테이너에 env var 로 전달하는 파이프라인이다. 수동으로 EB 환경변수를 세팅할 필요가 없다.

데이터 흐름

```
Terraform apply
| publish
▼
SSM Parameter Store /otk-pwrs/{stage}/...
|
| (1) buildspec 이 aws ssm get-parameter 로 조회
▼
.ebextensions/env.config (artifact 에 포함)
|
| (2) CodePipeline Deploy → EB 가 option_settings 적용
```

▼
 EB application environment
 |
 | (3) 컨테이너 환경변수로 주입
 ▼
 Spring Boot \${RDS_HOST}, \${REDIS_ENDPOINT}, \${RDS_SECRET_ARN} ...

SSM 경로 규약

`/{{project}}/{{stage}}/{{category}}/{{key}}`

경로	타입	값 출처
<code>/otk-pwrs/dev/rds/host</code>	String	RDS Endpoint (aws_db_instance.main.address)
<code>/otk-pwrs/dev/rds/port</code>	String	RDS Port (5432)
<code>/otk-pwrs/dev/rds/username</code>	String	Master username (otkadmin)
<code>/otk-pwrs/dev/rds/secret-arn</code>	String	RDS master user secret ARN (Secrets Manager)
<code>/otk-pwrs/dev/redis/endpoint</code>	String	ElastiCache primary endpoint (standalone)
<code>/otk-pwrs/dev/redis/port</code>	String	Redis Port (6379)

비밀(Password) 은 SSM에 저장하지 않는다. RDS master user 비밀번호는 Secrets Manager에서 RDS가 자동 관리하는 secret을 그대로 사용하며, 그 ARN만 non-secret으로 publish한다. 앱이 필요 시 `RDS_SECRET_ARN` 을 이용해 런타임에 Secrets Manager 를 직접 조회한다. Prod 는 `/otk-pwrs/prod/...` 로 publish 되며, 나머지 구조는 동일하다.

.ebextensions/env.config 생성

buildspec 의 `post_build` 단계에서 SSM 값을 조회해 artifact 에 포함할 `.ebextensions/env.config` 를 렌더링한다. EB 는 배포 시 이 파일의 `option_settings` 를 현재 환경 설정 위에 덧씌우므로, Terraform 의 `ignore_changes=[setting]` 정책과 충돌 없이 env var 를 갱신할 수 있다.

```
option_settings:
  aws:elasticbeanstalk:application:environment:
    STAGE: "dev"
    PROJECT: "otk-pwrs"
    AWS_REGION: "ap-northeast-2"
    RDS_HOST: "dev-otk-pwrs-db.xxxx.ap-northeast-2.rds.amazonaws.com"
    RDS_PORT: "5432"
    RDS_USERNAME: "otkadmin"
    RDS_SECRET_ARN: "arn:aws:secretsmanager:ap-northeast-2:<ACCOUNT_ID>:secret:rds!db-xxxx"
    REDIS_ENDPOINT: "dev-otk-pwrs-redis.xxxx.apn2.cache.amazonaws.com"
    REDIS_PORT: "6379"
```

buildspec 예시는 섹션 10.1 의 `buildspec.yml` 을 참고한다.

필요한 IAM 권한

역할	권한	용도
<code>dev-otk-pwrs-backend-build-role</code>	<code>ssm:GetParameter*</code> (Parameter Store 경로 한정)	빌드 시 SSM 값 조회

dev-otk-pwrs-eb-ec2-role	ssm:GetParameter* + secretsmanager:GetSecretValue + kms:Decrypt	런타임에 직접 Secret 조회(필요시)
--------------------------	---	------------------------

상세 정책 JSON 은 섹션 3.2, 3.3 참고.

값 갱신

- **RDS/Redis endpoint** 가 재생성되어 바뀌는 경우: `terraform apply` 만 실행하면 SSM 값이 갱신된다. 다음 Backend 배포부터 새 값이 `.ebextensions/env.config` 에 반영된다.
- 즉시 반영이 필요한 경우: CodePipeline 의 Backend pipeline 을 **Release change** 로 재실행한다.

6. ACM 인증서

콘솔: ACM (Certificate Manager) > Request certificate > Request a public certificate

Dev (ap-northeast-2):

설정	값
Domain names	dev-powersalesapi.otoki.com
Validation	DNS validation

Prod (ap-northeast-2):

설정	값
Domain names	powersalesapi.otoki.com
Validation	DNS validation

DNS validation을 선택하면 ACM이 CNAME 레코드를 제시한다. Route 53 호스팅 영역에 해당 CNAME을 등록하면 인증서가 발급된다. Stage별로 인증서를 분리하여 갱신/삭제 시 상호 영향을 방지한다.

인프라 ↔ 앱 작업자 핸드오프

7. GitHub Connection & CodePipeline Role

이 섹션은 앱 작업자와 인프라 작업자가 협업하여 진행한다.

7.1 GitHub Connection (otk-pwrs-github) — 앱 작업자

콘솔: CodePipeline > Settings > Connections > Create connection

설정	값
Provider	GitHub
Connection name	otk-pwrs-github

1. **Install a new app** 클릭 → GitHub 앱 설치 → 리포지토리 액세스 허용
2. 연결 상태가 **Available**로 변경되면 완료
3. Connection ARN 확인: `arn:aws:codeconnections:ap-northeast-2:<ACCOUNT_ID>:connection/<CONNECTION_ID>`

Dev/Prod 공용으로 사용한다. Connection은 GitHub 계정 단위이므로 하나만 생성하면 된다.

7.2 CodePipeline Role (dev-otk-pwrs-pipeline-role) — 인프라 작업자

콘솔: IAM > Roles > Create role

1. Trusted entity type: **Custom trust policy**
2. 아래 신뢰 정책 JSON 입력:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "codepipeline.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

3. 권한 정책 선택: **AdministratorAccess-AWSElasticBeanstalk** (AWS 관리형 정책)
4. Role name: dev-otk-pwrs-pipeline-role
5. 역할 생성 후 > 권한 탭 > 권한 추가 > 인라인 정책 생성 > JSON
6. 인라인 정책 이름: dev-otk-pwrs-pipeline-policy

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "GitHubConnection",
      "Effect": "Allow",
      "Action": [
        "codeconnections:UseConnection",
        "codeconnections:GetConnection",
        "codeconnections:GetConnectionToken",
        "codestar-connections:UseConnection",
        "codestar-connections:GetConnection",
        "codestar-connections:GetConnectionToken"
      ],
      "Resource": [
        "arn:aws:codeconnections:ap-northeast-2:<ACCOUNT_ID>:connection/*",
        "arn:aws:codestar-connections:ap-northeast-2:<ACCOUNT_ID>:connection/*"
      ]
    },
    {
      "Sid": "PipelineArtifacts",
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:GetObjectAcl",
        "s3:PutObject",
        "s3:GetBucketAcl",
        "s3:GetBucketLocation",
        "s3:GetBucketVersioning",
        "s3:ListBucket"
      ]
    }
  ]
}
```

```
    ],
    "Resource": [
      "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts",
      "arn:aws:s3:::dev-otk-pwrs-pipeline-artifacts/*"
    ]
  },
  {
    "Sid": "CodeBuildStart",
    "Effect": "Allow",
    "Action": ["codebuild:BatchGetBuilds", "codebuild:StartBuild"],
    "Resource": "arn:aws:codebuild:ap-northeast-2:<ACCOUNT_ID>:project/dev-otk-pwrs-backend-build"
  },
  {
    "Sid": "SNSApproval",
    "Effect": "Allow",
    "Action": "sns:Publish",
    "Resource": "arn:aws:sns:ap-northeast-2:<ACCOUNT_ID>:dev-otk-pwrs-alerts"
  }
]
}
```

앱 작업자 영역

8. ECR (Container Registry)

콘솔: ECR > Repositories > Create repository

설정	Dev	Prod
Repository name	dev-otk-pwrs	prod-otk-pwrs
Image tag mutability	Mutable	Immutable (prod 권장)
Scan on push	Enabled	Enabled
Encryption	AES-256	AES-256

이미지 태그: {commit-sha-8자}-{timestamp} + latest

Lifecycle Policy:

- 리포지토리 선택 > Lifecycle policy > Create rule
- 이미지 태그 상태: 모두 선택
- Match criteria: Image count more than 10
- Action: Expire

9. Elastic Beanstalk (Backend)

10.1 Application 생성

콘솔: Elastic Beanstalk > Applications > Create application

설정	Dev	Prod
Application name	dev-otk-pwrs-backend	prod-otk-pwrs-backend

Description	otoki backend API (dev)	otoki backend API (prod)
-------------	-------------------------	--------------------------

10.2 Environment 생성

콘솔: Application 내 > Create environment

기본 설정

설정	값
Environment name	dev-otk-pwrs-backend-env
Environment tier	Web server environment
Platform	Docker
Platform branch	Docker running on 64bit Amazon Linux 2023

Service access (서비스 역할)

설정	값
Service role	aws-elasticbeanstalk-service-role
EC2 key pair	(비상용으로 생성해 두고 안전하게 보관 — 평상시 Session Manager 사용)
EC2 instance profile	dev-otk-pwrs-eb-ec2-role

Networking & Database

설정	값
VPC	dev-otk-pwrs-vpc
Instance subnets	dev-otk-pwrs-app-a, dev-otk-pwrs-app-c (App Private 서브넷)
Database	(EB 연동 DB 사용하지 않음 — 별도 RDS는 Data 서브넷에 배치)

Instance traffic and scaling

설정	Dev	Prod
Environment type	Load balanced	Load balanced
Instance type	t3.small	t3.small
Min instances	1	2
Max instances	1	4

Instance type t3.small 선택 이유: Spring Boot 4.x + JVM (JPA, Redis, Secrets Manager) 의 메타스페이스/힙 및 EB-CloudWatch agent 오버헤드가 1 GB (t3.micro) 를 거의 포화시킨다 (~91%). t3.small (2 GB) 로 올려 JVM 에 안정적인 헤드를 확보. | Load balancer type | Application Load Balancer | Application Load Balancer | | Load balancer subnets | Public 서브넷 (dev-otk-pwrs-public-a , dev-otk-pwrs-public-c) | Public 서브넷 (prod-otk-pwrs-public-a , prod-otk-pwrs-public-c) | | Listener | 80 (+ 443 with ACM cert) | 80 (+ 443 with ACM cert) |

Dev/Prod 모두 Load balanced로 구성한다. Dev는 인스턴스를 1개로 고정하여 비용을 절약하면서도 ALB를 통한 HTTPS 지원 및 안정적인 API endpoint(dev-powersalesapi.codapt.kr) 를 확보한다. 이 서버도메인은 SAP 등 서버→서버 외부 소비자용 직접 경로로 유지되며, 웹 브라우저는 CloudFront /api/* path routing 을 거쳐 same-origin 으로 호출한다 (CORS 불필요).

Configure updates, monitoring, and logging

설정	Dev	Prod
Deployment policy	All at once	Rolling with additional batch
Health reporting	Basic	Enhanced
Log streaming	CloudWatch Logs 활성화	CloudWatch Logs 활성화

Security Group

- Instance security group: `dev-otk-pwrs-eb-sg`
- (Load balanced 환경의 경우) ALB security group: `dev-otk-pwrs-alb-sg`

10.3 Docker 설정 — `Dockerrun.aws.json`

`Dockerrun.aws.json` 은 저장소에 두지 않는다. `backend/buildspec.yml` 의 `post_build` 단계에서 CodeBuild 가 다음 형태로 동적으로 생성하고 artifact 로 EB 에 전달한다.

```
{
  "AWSEBDockerrunVersion": "1",
  "Image": {
    "Name": "${IMAGE_URI}",
    "Update": "true"
  },
  "Ports": [
    { "ContainerPort": 8080 }
  ]
}
```

`IMAGE_URI` 는 `${ACCOUNT_ID}.dkr.ecr.${AWS_DEFAULT_REGION}.amazonaws.com/${IMAGE_REPO_NAME}:${IMAGE_TAG}` 로 조합된다. Account ID 는 `aws sts get-caller-identity` 로 런타임 조회하고, `IMAGE_REPO_NAME` 은 CodeBuild 환경변수(Terraform 이 주입) 로 결정된다. 따라서 계정·스테이지별로 다른 값이 자동 반영된다.

9.4 환경변수 설정

콘솔에서 직접 설정하지 않는다. 섹션 "앱 구성 전달 (SSM Parameter Store)" 에 따라 **buildspec** 이 생성한 `.ebextensions/env.config` 가 배포 시 `aws:elasticbeanstalk:application:environment` option_settings 를 덮어써 컨테이너에 env var 를 주입한다.

주입되는 env var 목록(자동):

키	출처
<code>SPRING_PROFILES_ACTIVE</code>	하드코딩 aws (buildspec)
<code>STAGE</code>	CodeBuild env (dev / prod)
<code>PROJECT</code>	CodeBuild env (otk-pwrs)
<code>AWS_REGION</code>	CodeBuild env (ap-northeast-2)
<code>RDS_HOST</code>	SSM /otk-pwrs/{stage}/rds/host
<code>RDS_PORT</code>	SSM /otk-pwrs/{stage}/rds/port
<code>RDS_USERNAME</code>	SSM /otk-pwrs/{stage}/rds/username
<code>RDS_SECRET_ARN</code>	SSM /otk-pwrs/{stage}/rds/secret-arn

REDIS_ENDPOINT	SSM /otk-pwrs/{stage}/redis/endpoint
REDIS_PORT	SSM /otk-pwrs/{stage}/redis/port

비밀번호는 env var 로 주입되지 않는다. 앱이 `RDS_SECRET_ARN` 을 이용해 Secrets Manager 에서 런타임 조회한다 (`spring-cloud-aws-starter-secrets-manager`). EB EC2 역할(3.2) 에 필요한 권한이 부여되어 있다. Spring Boot 와이어랑: `application.properties` 는 `baseline+local`, `application-aws.properties` 가 `SPRING_PROFILES_ACTIVE=aws` 일 때 DB/Redis/Secrets Manager import 를 활성화한다. 의존성은 `spring-boot-starter-data-jpa + postgresql`, `spring-boot-starter-data-redis`, `io.awspring.cloud:spring-cloud-aws-starter-secrets-manager` (BOM 4.0.0). Terraform 의 `aws_elastic_beanstalk_environment.backend` 리소스는 `ignore_changes=[setting]` 이므로 `.ebextensions` 가 채우는 값과 충돌하지 않는다.

9.5 Health Check 설정

콘솔: EB > Environment > Configuration > Load balancer > Edit

- Health check path: `/actuator/health`

`spring-boot-starter-actuator` 의존성 필요. `/actuator/health` 는 DB (HikariCP), Redis (Lettuce) auto-configured health indicator 를 포함하며, `management.endpoint.health.show-details=always` 로 `components.db.status`, `components.redis.status` 까지 노출된다. **EB ALB 헬스체크 전용 경로**로 사용한다. 외부(웹/SAP) 노출용 상태 엔드포인트는 별도로 `/api/health` (`HealthController`) 를 통해 제공하며, 내부적으로 `HealthEndpoint` 를 재사용해 동일 형식 JSON 을 반환한다. CloudFront `/api/*` path routing 으로 브라우저는 same-origin 호출 → **CORS 불필요**. Dev/Prod 모두 ALB를 통한 health check를 수행한다.

9.6 HTTPS (ALB Listener 443) 설정

콘솔: EB > Environment > Configuration > Load balancer > Edit

Listener 추가:

설정	Dev	Prod
Port	443	443
Protocol	HTTPS	HTTPS
SSL cert	ACM 인증서 (섹션 6에서 발급한 Dev용)	ACM 인증서 (섹션 6에서 발급한 Prod용)
SSL policy	ELBSecurityPolicy-TLS13-1-2-2021-06	ELBSecurityPolicy-TLS13-1-2-2021-06

ALB가 TLS를 종료하고 백엔드(EC2/Docker)에는 HTTP(:80)로 전달하므로 애플리케이션 측 변경은 불필요하다. 기존 80 Listener는 유지하되, HTTP → HTTPS 리다이렉트가 필요하다면 EC2 > Load Balancers에서 80 Listener의 기본 액션을 `Redirect to HTTPS 443` 으로 변경한다.

9.7 배포 검증

- EB 환경 생성 완료 후 Environment URL 확인
- `http://<ENV_URL>/actuator/health` 접속하여 `{"status":"UP"}` 확인
- RDS 연결 확인: 애플리케이션 로그에서 DB 커넥션 에러 없는지 확인

10. CI/CD — Backend (CodePipeline + CodeBuild)

GitHub push → CodePipeline → [Source] → [Build: CodeBuild] → [Deploy: EB]

CodeBuild는 Docker 빌드 + ECR Push만 담당하고, EB 배포는 CodePipeline의 Deploy 스테이지가 처리한다.

Pipeline Artifact S3 버킷(`dev-otk-pwrs-pipeline-artifacts`)은 인프라 작업자가 4.4에서 생성한다.

10.1 Backend CodeBuild 프로젝트 생성

콘솔: CodeBuild > Build projects > Create build project

CodeBuild는 CodePipeline에서 호출되므로 **Webhook**을 설정하지 않는다.

프로젝트 설정

항목	값
Project name	dev-otk-pwrs-backend-build

소스

항목	값
Source provider	GitHub (CodeConnections 사용)
Connection	otk-pwrs-github (6.1에서 생성 완료)
Repository	codapt/otoki-code-build
Branch	dev
Buildspec	buildspec 파일 사용 — Buildspec name: buildspec.yml

CodePipeline에서 호출 시 소스가 오버라이드되지만, "소스 없음"을 선택하면 buildspec 파일 참조가 불가하므로 GitHub 소스를 지정한다. Webhook은 설정하지 않는다 — CodePipeline이 트리거를 관리한다.

환경

항목	값
Environment image	Managed image
Operating system	Amazon Linux
Runtime	Standard
Image	aws/codebuild/amazonlinux-x86_64-standard:6.0
Compute	BUILD_GENERAL1_MEDIUM (7GB RAM, 4 vCPU)
Privileged	체크 (Docker 빌드 필수)
Service role	Existing service role → dev-otk-pwrs-backend-build-role

주의: Privileged mode를 반드시 활성화해야 Docker 빌드가 가능합니다. BUILD_GENERAL1_MEDIUM 을 권장합니다 — Gradle + Kotlin 빌드는 메모리를 많이 사용합니다.

환경변수

변수	값	용도
AWS_DEFAULT_REGION	ap-northeast-2	ECR 로그인, SSM 조회
IMAGE_REPO_NAME	dev-otk-pwrs	ECR 리포지토리 이름
STAGE	dev / prod	.ebextensions/env.config 의 STAGE 값
PROJECT	otk-pwrs	.ebextensions/env.config 의 PROJECT 값

PARAM_PREFIX	/otk-pwrs/dev	buildspec 이 SSM Parameter Store 조회 시 사용할 경로 prefix
--------------	---------------	--

EB 관련 환경변수(`EB_APP_NAME` , `EB_ENV_NAME`)는 CodePipeline Deploy 스테이지에서 처리하므로 불필요.

buildspec.yml (Backend)

backend/buildspec.yml (저장소 내 경로). 스테이지-계정 종속 값은 buildspec 에 하드코딩하지 않고, 위 "환경변수" 표의 값들을 모두 CodeBuild 프로젝트 env (Terraform `cicd.tf` 가 주입) 로 전달한다.

```
version: 0.2

# AWS_DEFAULT_REGION, IMAGE_REPO_NAME, STAGE, PROJECT, PARAM_PREFIX 는
# CodeBuild 프로젝트 환경변수로 Terraform 이 주입한다.

phases:
  pre_build:
    commands:
      - echo "Logging in to Amazon ECR..."
      - ACCOUNT_ID=$(aws sts get-caller-identity --query Account --output text)
      - ECR_URI="${ACCOUNT_ID}.dkr.ecr.${AWS_DEFAULT_REGION}.amazonaws.com"
      - aws ecr get-login-password --region $AWS_DEFAULT_REGION | docker login --username AWS --
password-stdin $ECR_URI
      - IMAGE_TAG="${CODEBUILD_RESOLVED_SOURCE_VERSION:0:8}-${date +%Y%m%d%H%M%S}"
      - IMAGE_URI="${ECR_URI}/${IMAGE_REPO_NAME}:${IMAGE_TAG}"
      - echo "Image will be tagged as ${IMAGE_URI}"

  build:
    commands:
      - echo "Building Docker image..."
      - docker build -t $IMAGE_REPO_NAME:$IMAGE_TAG ./backend
      - docker tag $IMAGE_REPO_NAME:$IMAGE_TAG $IMAGE_URI
      - docker tag $IMAGE_REPO_NAME:$IMAGE_TAG "${ECR_URI}/${IMAGE_REPO_NAME}:latest"

  post_build:
    commands:
      - echo "Pushing Docker image to ECR..."
      - docker push $IMAGE_URI
      - docker push "${ECR_URI}/${IMAGE_REPO_NAME}:latest"
      - echo "Image pushed successfully - ${IMAGE_URI}"
      - echo "Copying .ebextensions for EB deployment..."
      - cp -r backend/.ebextensions .ebextensions
      - |
        # Fetch dynamic app config from SSM Parameter Store and render
        # .ebextensions/env.config so EB injects them as env vars at deploy time.
        echo "Fetching app config from SSM (${PARAM_PREFIX})..."
        get_param() {
          aws ssm get-parameter --name "$1" --query 'Parameter.Value' --output text
        }
        RDS_HOST=$(get_param "${PARAM_PREFIX}/rds/host")
        RDS_PORT=$(get_param "${PARAM_PREFIX}/rds/port")
        RDS_USERNAME=$(get_param "${PARAM_PREFIX}/rds/username")
        RDS_SECRET_ARN=$(get_param "${PARAM_PREFIX}/rds/secret-arn")
        REDIS_ENDPOINT=$(get_param "${PARAM_PREFIX}/redis/endpoint")
        REDIS_PORT=$(get_param "${PARAM_PREFIX}/redis/port")
        cat > .ebextensions/env.config <<EOFENV
        option_settings:
```

```

aws:elasticbeanstalk:application:environment:
  STAGE: "${STAGE}"
  PROJECT: "${PROJECT}"
  AWS_REGION: "${AWS_DEFAULT_REGION}"
  RDS_HOST: "${RDS_HOST}"
  RDS_PORT: "${RDS_PORT}"
  RDS_USERNAME: "${RDS_USERNAME}"
  RDS_SECRET_ARN: "${RDS_SECRET_ARN}"
  REDIS_ENDPOINT: "${REDIS_ENDPOINT}"
  REDIS_PORT: "${REDIS_PORT}"
EOFENV
- echo "Generating Dockerrun.aws.json for EB deployment..."
- |
cat > Dockerrun.aws.json <<EOFDR
{
  "AWSEBDockerrunVersion": "1",
  "Image": {
    "Name": "${IMAGE_URI}",
    "Update": "true"
  },
  "Ports": [
    { "ContainerPort": 8080 }
  ]
}
EOFDR

artifacts:
  files:
    - Dockerrun.aws.json
    - .ebextensions/**/*

```

`artifacts` 섹션에 `Dockerrun.aws.json` 과 `.ebextensions/**/*` 를 모두 포함시킨다. CodePipeline Deploy 스테이지가 EB 에 전달하면 EB 가 `.ebextensions` 의 `option_settings` 를 현재 환경 설정에 덧씌워 `env var` 로 주입한다.

10.2 CodePipeline 생성 (Dev)

콘솔: CodePipeline > Pipelines > Create pipeline

Step 1 — Pipeline settings

설정	값
Pipeline name	dev-otk-pwrs-backend-pipeline
Pipeline type	V2
Execution mode	Queued
Service role	Existing service role → dev-otk-pwrs-pipeline-role
Artifact store	Custom location → dev-otk-pwrs-pipeline-artifacts
Encryption key	Default AWS Managed Key (SSE-S3, KMS 사용 안 함)

Step 2 — Source stage

설정	값
----	---

Source provider	GitHub(GitHub 앱을 통해)
Connection	otk-pwrs-github (6.1에서 생성 완료)
Repository name	codapt/otoki-code-build
Default branch	dev
Output artifact format	CodePipeline default

Connection은 6.1에서 이미 생성했으므로 드롭다운에서 `otk-pwrs-github` 를 선택한다.

Webhook 이벤트

설정	값
Webhook	체크 (푸시 및 풀 요청 이벤트)
이벤트 유형	푸시
필터 유형	브랜치
브랜치 패턴	dev
파일 경로	backend/**

`dev` 브랜치에 푸시되고, `backend/` 하위 파일이 변경된 경우에만 파이프라인이 트리거된다.

Step 3 — Build stage

설정	값
Build provider	AWS CodeBuild
Project name	dev-otk-pwrs-backend-build
Input artifacts	SourceArtifact
Output artifacts	BuildArtifact

Step 4 — Deploy stage

설정	값
Deploy provider	AWS Elastic Beanstalk
Application name	dev-otk-pwrs-backend
Environment name	dev-otk-pwrs-backend-env
Input artifacts	BuildArtifact

CodeBuild가 출력한 `Dockerrun.aws.json` 이 포함된 BuildArtifact를 EB에 전달한다. EB는 이 파일을 기반으로 ECR에서 Docker 이미지를 pull하여 배포한다.

검증

1. CodePipeline 콘솔 > 파이프라인 선택 > **Release change** (수동 트리거)
2. Source → Build → Deploy 각 스테이지 성공 여부 확인
3. Build 스테이지: CodeBuild 로그에서 Docker 빌드 + ECR push 확인
4. Deploy 스테이지: EB 환경에서 배포 상태 확인
5. `http://<ENV_URL>/actuator/health` 접속하여 `{"status":"UP"}` 확인

10.3 Prod CI/CD (CodePipeline)

Prod는 별도 CodePipeline + CodeBuild로 구성한다. Dev와 동일하되 아래 항목이 다르다.

항목	Dev	Prod
파이프라인	dev-otk-pwrs-backend-pipeline	prod-otk-pwrs-backend-pipeline
소스 브랜치	dev	main
CodeBuild 프로젝트	dev-otk-pwrs-backend-build	prod-otk-pwrs-backend-build
Service role	dev-otk-pwrs-pipeline-role	prod-otk-pwrs-pipeline-role
EB Application	dev-otk-pwrs-backend	prod-otk-pwrs-backend
EB Environment	dev-otk-pwrs-backend-env	prod-otk-pwrs-backend-env
환경변수	dev 리소스 참조	prod 리소스 참조

Prod 파이프라인은 수동 승인 스테이지를 추가한다:

Source → Build → Approval → Deploy

Approval 스테이지 설정

Build와 Deploy 사이에 **Manual approval** 액션을 추가:

설정	값
Action name	ManualApproval
Action provider	Manual approval
SNS topic ARN	arn:aws:sns:ap-northeast-2:<ACCOUNT_ID>:prod-otk-pwrs-alerts
Comments	Prod 배포를 승인하시겠습니까?

빌드 성공 후 SNS 알림이 전송되고, 콘솔에서 승인해야 Deploy 스테이지가 진행된다.

11. DNS 레코드 — 인프라 작업자

도메인이 있는 경우에만 해당한다. 이미 등록된 도메인의 호스팅 영역을 사용한다. 앱 작업자가 EB 환경 URL을 전달하면, 인프라 작업자가 아래 레코드를 등록한다.

콘솔: Route 53 > Hosted zones > otoki.com > Create record

Record	Type	Alias Target
dev-powersalesapi.otoki.com	A (Alias)	Dev EB 환경 URL
powersalesapi.otoki.com	A (Alias)	Prod EB 환경 URL

12. CloudWatch (모니터링 & 로그)

12.1 로그 그룹

EB와 CodeBuild는 자동으로 로그를 전송한다.

로그 그룹	소스
/aws/elasticbeanstalk/dev-otk-pwrs-backend-env/*	EB 인스턴스 로그
/aws/codebuild/dev-otk-pwrs-backend-build	Backend CodeBuild
/aws/rds/instance/dev-otk-pwrs-db/postgresql	RDS 로그 (활성화 필요)

12.2 알람 설정

Stage	적용 여부	비고
Dev	미설정	비용 절감 및 알람 노이즈 방지를 위해 생략
Prod	설정	운영 장애 감지를 위해 필수

Prod 알람 구성

콘솔: CloudWatch > Alarms > Create alarm

SNS 토픽 먼저 생성:

- Name: prod-otk-pwrs-alerts
- 이메일 구독 추가

권장 알람:

알람	지표	조건
EB 환경 상태	EnvironmentHealth	Degraded 또는 Severe
RDS CPU	CPUUtilization	> 80% (5분)
RDS 스토리지	FreeStorageSpace	< 5 GB
ALB 5XX 에러	HTTPCode_ELB_5XX_Count	> 10 (5분)

13. DB 접속 (SSM 포트 포워딩)

Bastion 서버 없이 SSM Session Manager를 통해 RDS에 접속한다. EB EC2 역할에 AmazonSSMManagedInstanceCore 가 이미 포함되어 있으므로 별도 인프라 추가 없이 사용 가능하다.

사전 준비: 로컬에 [Session Manager Plugin](#)을 설치한다.

1단계 — EB 인스턴스 ID 확인:

콘솔: EC2 > Instances > Project: otk-pwrs , Stage: dev 태그로 필터링 → Instance ID 복사

2단계 — 포트 포워딩 시작:

```
aws ssm start-session \
  --target <EB-INSTANCE-ID> \
  --document-name AWS-StartPortForwardingSessionToRemoteHost \
  --parameters '{"host": ["<RDS_ENDPOINT>"], "portNumber": ["5432"], "localPortNumber": ["5432"]}'
```

3단계 — DB 클라이언트 연결:

항목	값
----	---

Host	localhost
Port	5432
Database	otoki
Username	otkadmin
Password	(Secrets Manager 에서 조회 — 아래 참고)

Password 조회 (Secrets Manager):

```
# RDS master user secret ARN 은 SSM Parameter Store 에서도 조회 가능
SECRET_ARN=$(aws ssm get-parameter --name /otk-pwrs/dev/rds/secret-arn --query 'Parameter.Value' --
output text)
aws secretsmanager get-secret-value --secret-id "$SECRET_ARN" \
--query 'SecretString' --output text | jq -r .password
```

RDS 는 `manage_master_user_password = true` 로 설정되어 있어 비밀번호가 Secrets Manager 에 자동 저장·순환(옵션) 된다. 콘솔에서 는 RDS > Databases > `dev-otk-pwrs-db` > Configuration 탭의 **Master credentials ARN** 링크로도 이동할 수 있다. 기본 유효 타임아웃은 20분이다. 변경하려면 Systems Manager > Session Manager > Preferences에서 최대 60분까지 설정 가능하다.

14. Dev vs Prod 차이점 요약

항목	Dev	Prod
VPC CIDR	10.0.0.0/16	10.1.0.0/16
RDS 인스턴스	db.t4g.micro, Single-AZ	db.t4g.small, Single-AZ
RDS 삭제 방지	Off	On
RDS 백업 보존	3일	14일
ElastiCache	cache.t4g.micro (데모)	cache.t4g.micro (데모, 필요 시 업그레이드)
ECR Tag 변경	Mutable	Immutable
EB 인스턴스	t3.small, Load balanced (1-1)	t3.small, Auto Scaling (2-4)
EB 배포 방식	All at once	Rolling with additional batch
CI/CD 브랜치	dev (push 자동)	main (push + 수동 승인)
SNS 알림 토픽	미설정	prod-otk-pwrs-alerts (이메일 구독)
CloudWatch 알람	미설정	전체 구성 (12.2)
커스텀 도메인	선택	필수
예산 월 비용	~\$50-80	~\$200-400

부록

자주 쓰는 AWS CLI 명령


```
# ECR 로그인
aws ecr get-login-password --region ap-northeast-2 | \
  docker login --username AWS --password-stdin <ACCOUNT_ID>.dkr.ecr.ap-northeast-2.amazonaws.com

# EB 환경 상태 확인
aws elasticbeanstalk describe-environment-health \
  --environment-name dev-otk-pwrs-backend-env --attribute-names All

# EB 로그 확인
aws elasticbeanstalk request-environment-info \
  --environment-name dev-otk-pwrs-backend-env --info-type tail

# RDS 접속 (로컬에서 SSH 터널 또는 Session Manager 필요)
# Private 서브넷이므로 직접 접속 불가
```

트러블슈팅

증상	원인	해결
EB에서 Docker pull 실패	EC2 인스턴스 프로필에 ECR 권한 없음	AmazonEC2ContainerRegistryReadOnly 정책 연결
EB 환경 Degraded	Health check 실패	/actuator/health 엔드포인트 확인, Security Group 확인
RDS 접속 불가	Security Group 미설정	dev-otk-pwrs-rds-sg에서 EB SG 인바운드 허용 확인
NAT Gateway 비용	불필요한 아웃바운드 트래픽	VPC Endpoint 추가 (S3, ECR) 고려

향후 고려사항

- **WAF**: Prod ALB 앞단에 WAF 적용 (DDoS, SQL Injection 방어)
- **VPC Endpoint**: S3, ECR용 VPC Endpoint로 NAT Gateway 비용 절감
- **Secrets Manager**: RDS 비밀번호를 Secrets Manager로 관리
- **Mobile**: API Gateway + Lambda 또는 기존 EB Backend API 공유
- **Web 인프라**: Frontend(S3 + CloudFront) 설정은 `infra-web.md` 참고